

Experiment 6

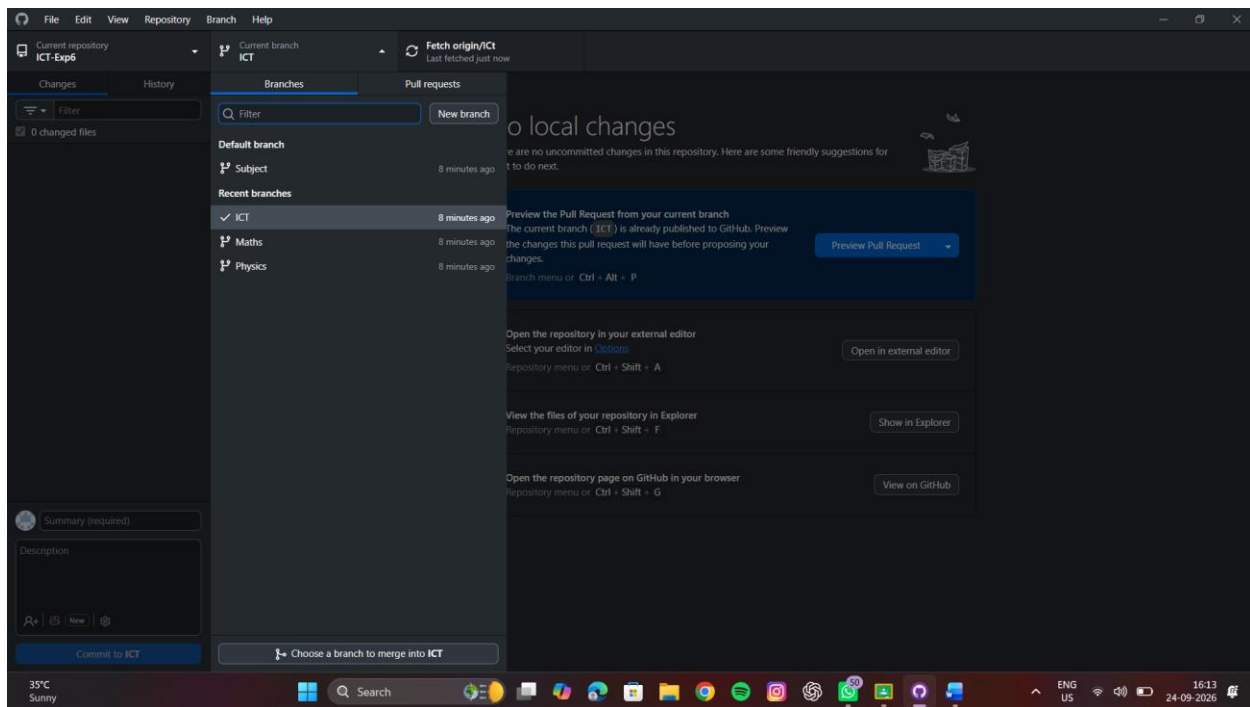
File Tracking, Commit Management, and Repository Maintenance Using Git

Theory

Git is a distributed version control system used to track, manage, and store changes made to files in a project. It maintains a complete history of modifications through commits, making it easier to monitor and restore previous versions when required. File tracking in Git is performed by adding files to the staging area using the `git add` command, while `git status` helps identify tracked, untracked, and modified files. Commit management is used to permanently save staged changes in the repository using the `git commit` command with a meaningful message. The `git log` command can be used to view the history of commits and understand the development of a project. Repository maintenance involves regularly checking the repository status, maintaining meaningful commit history, and removing unnecessary files or branches. Git provides an organized and reliable method for managing project files and their versions. It is also useful for collaborative software development because multiple changes can be managed systematically. Overall, Git helps improve code organization, version control, and project maintenance.

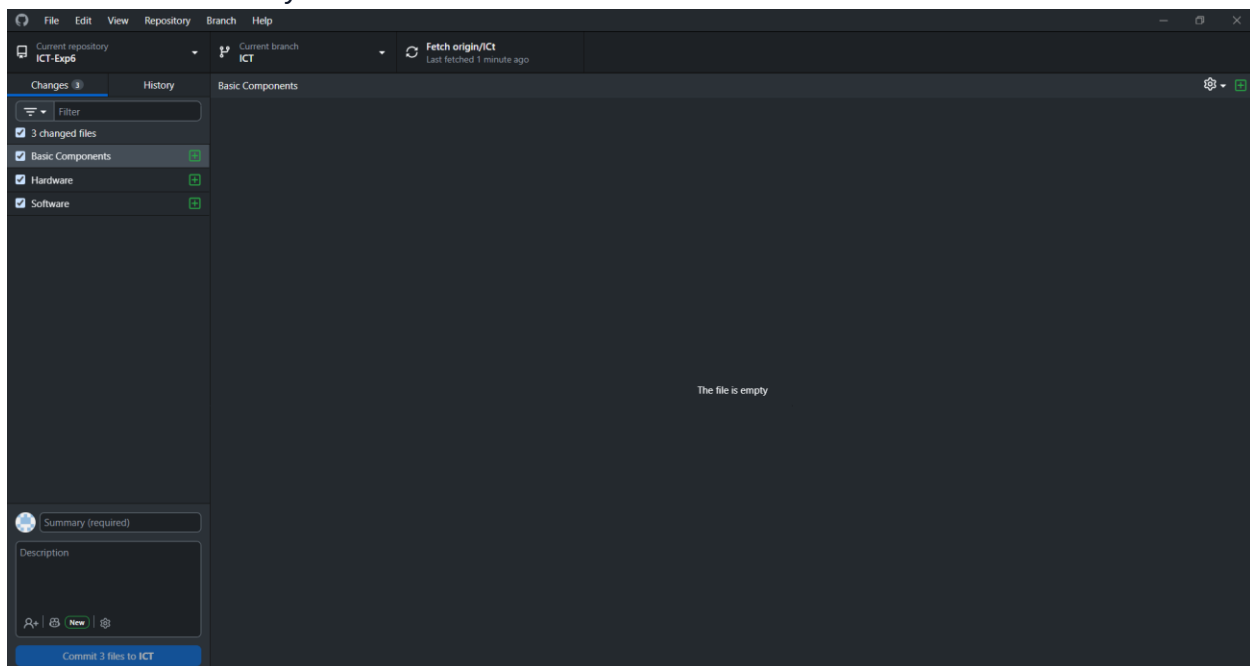
1. File Tracking & Working Tree State

1. Git manages state transitions between your active workspace, staging area (index), and committed history. Tracking keeps record of untracked additions, uncommitted file modifications, and staged units awaiting consolidation.
2. Workspace to index



2. Commit Management & History

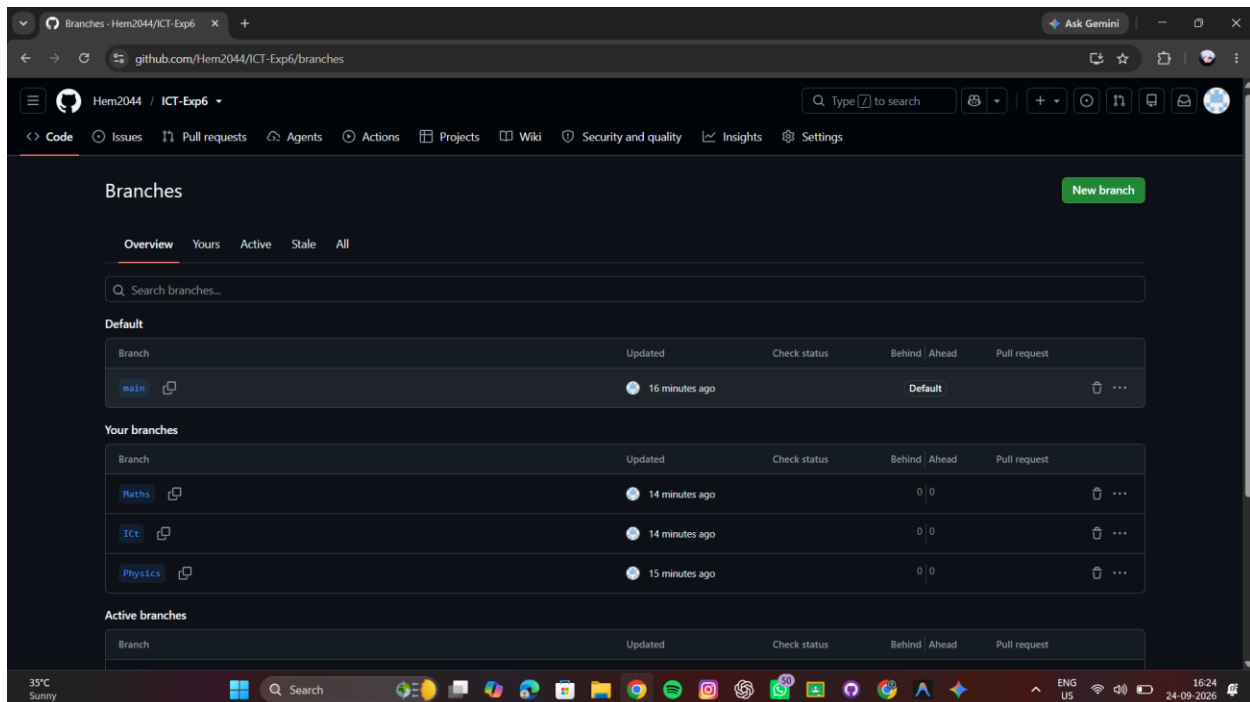
1. Commits represent cryptographically hashed, permanent snapshots of staged files accompanied by semantic descriptions. Rigorous commit practices ensure clear audit trails, effortless feature rollbacks, and coherent collaborative merges.
2. Add summary and description about your change and commit changes.
3. Commit history is visible on GitHub.com



```
.git > COMMIT_EDITMSG
1
2 # Please enter the commit message for your changes. Lines starting
3 # with '#' will be ignored, and an empty message aborts the commit.
4 #
5 # On branch ICT
6 # Your branch is up to date with 'origin/Ict'.
7 #
8 # Changes to be committed:
9 #   new file:   Hem
10 #   new file:   Student
11 #
12
```

3. Repository Maintenance & Health

1. Over extensive release cycles, repositories accumulate dangling objects, fragmented packfiles, and stale remote branches. Routine maintenance preserves snappy CLI performance, reclaims storage capacity, and guarantees object-database consistency.
2. Check periodically on github.



Link to access the repository : <https://github.com/Hem2044/ICT-Exp6.git>

Conclusion

The experiment demonstrates how Git can be used to track files, manage project commits, and maintain a repository. Using commands such as `git add`, `git commit`, `git status`, and `git log` helps maintain a clear history of project changes and makes project management more organized and reliable.